PIM-CCA: An Efficient PIM Architecture with Optimized Integration of Configurable Functional Units

Jeehyun Kim
Yonsei University
Seoul, Republic of Korea
jeehyun990@yonsei.ac.kr

Donghyeon Kim
Yonsei University
Seoul, Republic of Korea
dhkim9309@yonsei.ac.kr

Seokwon Kang
Yonsei University
Seoul, Republic of Korea
kswon0202@yonsei.ac.kr

Bongjoon Hyun
KAIST
Daejeon, Republic of Korea
bongjoon.hyun@kaist.ac.kr

Inho Lee
Hanyang University
Seoul, Republic of Korea
inholee@hanyang.ac.kr

Yongjun Park
Yonsei University
Seoul, Republic of Korea
yongjunpark@yonsei.ac.kr

Abstract

Processing-in-Memory (PIM) is a promising architecture for alleviating data movement bottlenecks by performing computations closer to memory. However, PIM workloads often encounter computational bottlenecks within the PIM itself. As these workloads become more compute-intensive by leveraging PIM's high internal bandwidth, a small set of hot code regions emerges as the primary performance bottleneck. Unfortunately, increasing the complexity of the PIM processor is difficult due to inherent memory constraints, such as area and power. Therefore, enhancing the computational capability of PIM while maintaining a lightweight design within limited silicon budgets remains highly challenging.

In this paper, we propose PIM-CCA, a novel PIM architecture that integrates a Configurable Compute Accelerator (CCA) to mitigate computational bottlenecks with minimal hardware overhead. The CCA-enabled PIM design allows for the flexible configuration of compute logic, enabling acceleration across diverse workloads. The PIM-CCA compiler constructs an instruction-level dataflow graph to identify hot and compute-bound regions and offload them to the CCA. Furthermore, we analyze the interaction between the PIM threading model and resource utilization to derive the optimal thread count for efficient CCA-enabled PIM usage. We implement PIM-CCA in a cycle-accurate simulator based on a commercially available PIM system, and evaluate it using 14 representative benchmarks. The experimental results show that PIM-CCA achieves up to 1.55× performance improvement over baseline PIM systems, with only 0.036% additional area overhead, based on P&R results with limited metal layers.

CCS Concepts

• **Hardware** → **Emerging architectures**; • **Computer systems organization** → *Reconfigurable computing*; • **Software and its engineering** → *Compilers*.

MICRO '25, Seoul, Republic of Korea

ACM ISBN 979-8-4007-1573-0/25/10
https://doi.org/10.1145/3725843.3756034

Keywords

Accelerators, Processing-in-memory, Reconfigurable architectures and systems, Memory technologies and architectures

ACM Reference Format:
Jeehyun Kim, Donghyeon Kim, Seokwon Kang, Bongjoon Hyun, Inho Lee, and Yongjun Park. 2025. PIM-CCA: An Efficient PIM Architecture with Optimized Integration of Configurable Functional Units. In *58th IEEE/ACM International Symposium on Microarchitecture (MICRO '25), October 18–22, 2025, Seoul, Republic of Korea.* ACM, New York, NY, USA, 15 pages. https://doi.org/10.1145/3725843.3756034

1 Introduction

As data-centric workloads continue to grow in size and depth across domains, the problem-solving paradigm is shifting from application-centric to a data-centric approaches [3, 5, 20, 22, 30]. Although computational capabilities have improved significantly through multi-core architectures, memory access remains the performance bottleneck because bandwidth cannot scale proportionally with density. Consequently, the memory wall problem is becoming increasingly critical, as all data must be transferred between memory and processor under the Von Neumann architecture [32].

Various architectural designs have been proposed to address these challenges, including Processing-in-Memory (PIM)[11, 12, 14, 19, 21, 25, 26, 45], Near-Data Processing (NDP)[1, 2, 10, 15, 36], and In-Storage Processing (ISP)[29, 31, 35, 41, 42]. Of these, PIM, originating from the idea of integrating compute capabilities into memory chips, is one of the most promising solutions to the memory wall problem. Although the computational logic in PIM is usually located close to, rather than directly within, memory cells, it facilitates high-bandwidth in-memory computation. Despite having relatively simple processing units, PIM offers clear advantages for memory-intensive workloads such as General Matrix-Vector Multiplication (GEMV). Leveraging this strength, recent studies have increasingly focused on PIM architectures [4, 17, 18, 33, 37, 40].

UPMEM introduced the first commercially available PIM-enabled Dual In-line Memory Module (DIMM) [9, 44]. This module integrates general-purpose pipeline processors (DRAM Processing Units, or DPUs) with DRAM banks, offering significantly higher bandwidth than traditional CPUs. The PIM module is provided in the form of a standard DDR4 DIMM, equipped with 128 fully independent DPUs. Each DPU is built upon a multi-stage pipeline design, which aims to overlap computation and memory access

for enhanced performance. Additionally, UPMEM offers high programmability through a well-designed programming model similar to CUDA/OpenCL. Thanks to this novel hardware design and robust software stack, workloads can be easily offloaded to and scaled on PIM. However, there are still several limitations in extending its applicability to broader domains and meeting future performance requirements.

[**1. Computational Limitations of PIM Processors**] First, PIM suffers from limited computational capability due to wiring and area constraints. Because the PIM processor is integrated within the memory chip, which must preserve sufficient capacity for data storage, the available area for computation logic is inherently limited. As a result, the processor design is necessarily simple, often lacking the capability to efficiently handle complex operations such as floating-point operations. Furthermore, PIM cannot typically benefit from high operating frequencies, as it shares system clocks with memory. Although memory-intensive workloads generally exhibit lower computational complexity, the increasing complexity of widely-used applications continues to drive the demand for higher compute capabilities within PIM. However, current PIM architectures face inherent limitations in meeting these demands due to their design constraints.

[**2. Changes in Characteristics inside PIM**] Furthermore, because offloaded workloads to PIM are not always purely memory-intensive, once inside the PIM architecture, they often shift towards being compute-intensive. While PIM leverages high memory bandwidth to reduce communication overhead, this advantage can inadvertently increase computational intensity. Traditional architectures address such challenges by enhancing processor performance through higher clock frequencies, multi-core designs, or vector-level parallelism (e.g., SIMD, SPMD). In PIM, however, these enhancements are constrained by the aforementioned architectural limitations. Moreover, the shift in characteristics and the primary computational bottlenecks are often workload-dependent. Therefore, designing a general-purpose, complex processor to handle all possible bottlenecks would be inefficient in the context of PIM. To overcome these challenges, a solution is needed that can efficiently address diverse computational bottlenecks while maintaining the strict frequency and area constraints of PIM architectures.

To efficiently mitigate computational bottlenecks in PIM workloads, it is also essential to selectively accelerate code regions that are both performance-critical and frequently executed. These regions, referred to as hot code, represent the primary bottlenecks. By identifying hot code regions, it is possible to offload or accelerate only the most critical bottlenecks within the limited capabilities. Furthermore, because hot code often appears as loops, repeated data transformations, or tight compute kernels, offloading them enables the system to sustain high throughput without deviating from the memory-centric paradigm. Thus, accurately identifying and selectively accelerating hot code regions is important to effectively utilize the full potential of constrained PIM resources.

One promising approach to address these challenges is the application of Configurable Compute Accelerators (CCAs) [6, 7] within PIM architectures. CCAs are specialized hardware units originally designed to enhance computational efficiency and flexibility for domain-specific workloads. Its core concept is to extend the instruction set with custom instructions tailored to specific tasks. This reconfigurable design improves efficiency across diverse workloads and accelerates performance-critical bottlenecks. Furthermore, CCAs can be more easily integrated into existing architectures than enhancing the main PIM processor or incorporating fixed-function accelerators, as its area can be carefully controlled and its design supports high modularity and adaptability through Custom Functional Units (CFUs). With these features, CCA becomes a compelling solution for improving compute power in resource-constrained PIM architectures. However, to fully exploit CCA, it is crucial to adopt PIM-specific design strategies and instruction offloading mechanisms that align with the memory-centric computing paradigm.

[**3. Balancing Tasklet Parallelism in PIM**] Another important consideration in PIM, such as UPMEM, is the use of software-managed threads (called *tasklets*). Since the DPU features an 11-stage pipeline, it can theoretically support up to 11 concurrent tasklets to fully utilize its stages. However, our observations show that frequent memory stalls (e.g., load/store) often limit performance scalability beyond a certain tasklet count. Increasing the number of tasklets incurs control-flow, scheduling, and synchronization overheads, which can offset the benefits of higher concurrency. Therefore, using fewer tasklets than the pipeline depth can achieve near-optimal performance while reducing management overheads. Although concurrency models and threading mechanisms may differ across architectures, our observations provide some valuable insights: balancing pipeline-level parallelism against overheads is crucial for maximizing throughput in PIM systems.

A CCA-enabled PIM architecture can mitigate these issues. By accelerating compute-bound hot code, CCA reduces the need for excessive tasklets, thereby improving resource efficiency in memory-bound regions. However, in a CCA-enabled PIM architecture, too few tasklets may under-utilize both the CCA and the DPU, whereas too many tasklets can cause high resource contention. Therefore, carefully tuning the tasklet count is essential in order to fully leverage both the high internal memory bandwidth and hardware acceleration, ultimately maximizing overall system performance.

In this paper, we propose ***PIM-CCA***, a novel design that integrates a CCA into existing PIM architectures. PIM-CCA first introduces a PIM-aware CCA design to accelerate computational bottlenecks within the PIM. It identifies hot code regions, which are performance bottlenecks that are executed iteratively, through instruction-level graph analysis, and offloads them to the CCA for the efficient acceleration of complex computations. Furthermore, we analyze the impact of tasklet count on the performance of the CCA-enabled PIM architecture and determine the best thread counts for effective resource utilization.

We implement PIM-CCA on a cycle-accurate PIM simulator (uPIMulator [16]), to evaluate its ability to improve compute intensity in memory-intensive workloads based on existing PIM architectures. Our proposed strategy for resolving computational bottlenecks via CCA is not limited to the UPMEM architecture and can be applied to other PIM designs as well.

This paper provides the following contributions:

- An in-depth analysis of compute intensity issues in memory-intensive workloads offloaded to PIM architectures.

- A novel design that integrates a CCA accelerator into existing PIM architectures.
- A hot code detection mechanism based on data dependency graph analysis, with instruction-level offloading of computational bottlenecks using a CCA.
- Optimization of tasklet counts for efficient utilization of the CCA-enabled PIM architecture.
- Evaluation of PIM-CCA on a cycle-accurate simulator, achieving up to 1.55x improvement with 0.036% hardware overhead over existing PIM designs.

2 Background & Motivation

2.1 PIM Architecture & uPIMulator

The basic concept of the PIM architecture is to perform computations within the same chip that contains memory. A variety of PIM architectures have been proposed, including AiM, Aquabolt-XL, and HBM-PIM [11, 12, 14, 19, 21, 23, 25, 26, 45]. UPMEM [9, 44] is one of the commercially accessible PIM solutions. Unlike memory-centric PIM designs, UPMEM has a more accelerator-like design by integrating general-purpose processors, offering greater programmability and flexibility. Although it has relatively low internal memory bandwidth, UPMEM compensates for this with greater computational capability per unit, making it a practical choice for a broader range of workloads.

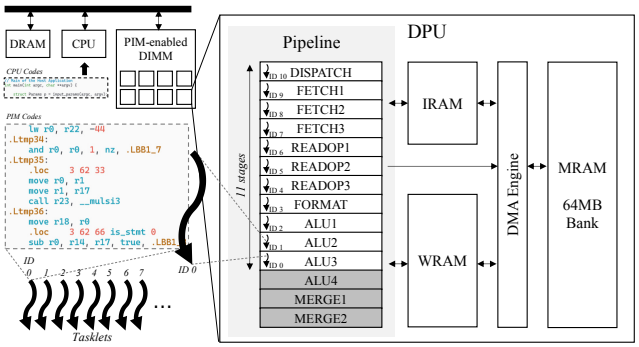


Figure 1: An UPMEM PIM architecture.

Figure 1 shows a high-level overview of UPMEM PIM architectures. UPMEM provides PIM solutions in the form of DIMM modules, with each module comprising 128 memory banks integrated with PIM units. Each PIM unit consists of a DPU, three types of memory, and a Direct Memory Access (DMA) engine. The DPU is a scalar processor with 14 pipeline stages, but only the final three stages (ALU4-MERGE1-MERGE2) can overlap with the DISPATCH and FETCH stages. This restricts instruction-level parallelism; therefore, each DPU supports up to 11 concurrently executing tasklets. The DMA engine enables the DPU to access 64 MB of Main Memory (MRAM), 64 KB of scratch pad memory (WRAM), and 24 KB of instruction memory (IRAM) directly. UPMEM also provides an open-source, full-stack SW solution including a C-like Single-Program Multiple-Data (SPMD) programming model, an LLVM-based compiler, and runtime libraries. Based on its architectural design, UPMEM is well-suited for memory-intensive and data-parallel workloads. Its software stack further supports the simple offloading of such workloads to the PIM units.

uPIMulator [16] is a flexible, highly accurate, cycle-level simulator based on the UPMEM architecture. It uses UPMEM's open-source SDK and LLVM-based compiler toolchain to translate source-level PIM programs into ISA-compatible binaries, which are then executed in a custom-designed, in-order DPU simulator. To overcome the limitations of the fixed UPMEM toolchain, uPIMulator integrates its own linker and assembler. This enables flexible mapping of code and data to memory, supporting exploration beyond the baseline UPMEM design.

As our primary focus is addressing the computational bottlenecks in offloaded workloads on PIM, we implemented and evaluated our insights using uPIMulator based on the UPMEM architecture.

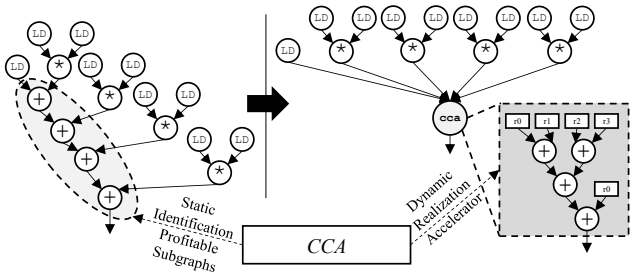


Figure 2: The fundamental concepts of CCA frameworks.

2.2 Configurable Compute Accelerator

CCAs are specialized hardware units designed to accelerate domain-specific computations by extending a processor's instruction set with custom instructions. They were originally proposed to improve the performance and energy efficiency of embedded systems. As shown in Figure 2, CCAs identify and offload critical dataflow subgraphs within applications into reconfigurable hardware composed of multiple CFUs. This strategy allows performance-critical operations to be accelerated without the significant increase of processor complexity or hardware overhead. Unlike fixed-function accelerators, CCAs are generated based on compiler guidance, and maintain programmability and adaptability across various workloads through a combination of static subgraph identification and dynamic mapping mechanisms.

The original CCAs [6, 7] were proposed from a transparent instruction set customization framework that integrates them seamlessly with general-purpose processors. It employs a hybrid model in which the compiler identifies candidate computation subgraphs, and the runtime system dynamically maps and executes them via a modular CCA subsystem. By enabling the reuse of CFUs across applications, as well as supporting wildcarding and subgraph sharing, CCAs provide high hardware utilization and flexibility. This makes the CCA concept particularly appealing in memory-constrained or power-sensitive environments, such as PIM, where integrating lightweight, adaptable accelerators can address both compute and memory bottlenecks efficiently.

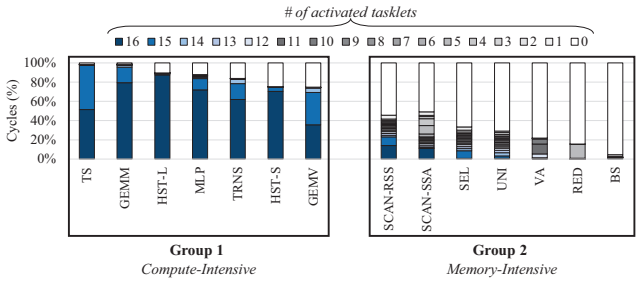


Figure 3: Breakdown of cycles based on the number of activated tasklets.

2.3 Compute & Memory Intensity inside PIM

Recent studies have shown that memory-intensive workloads can mitigate the memory wall effectively through PIM. However, we have observed that, as communication bottlenecks are alleviated by offloading tasks to PIM, some workloads exhibit a shift towards compute-intensive behavior.

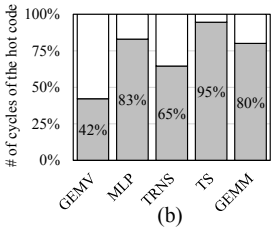
Figure 3 shows the distribution of execution cycles for each benchmark and the number of activated tasklets. This observation was made by examining the cycles on the PIM unit for each workload of PrIM benchmark [13] on the uPIMulator. While these workloads are originally memory-intensive on traditional processors, they exhibit different characteristics within PIM. The workloads in Group 1 change to compute-intensive behavior within the PIM, with most cycles involving substantially activated tasklets. These workloads benefit from increased memory bandwidth, shifting the primary bottleneck from memory to computation. Conversely, the workloads in Group 2 remain memory-bound. This is evidenced by the majority of cycles executing with no or few activated tasklets due to memory stalls. These workloads continue to suffer from memory bottlenecks, even with the high internal bandwidth.

As shown in Figure 3, most workloads alleviate memory bottlenecks through higher bandwidth, which leads to higher computational intensity. However, some workloads still suffer from communication bottlenecks, which remain unresolved even within memory-centric architectures. These diverse, imbalanced bottlenecks lead to inefficiencies and under-utilization of PIM resources, ultimately degrading performance. While traditional processors have addressed computational bottlenecks through complex hardware modifications such as multi-core designs, higher frequencies, and vector-level parallelism, improving the computational capability of PIM faces fundamental limitations. Due to the physical and thermal constraints of memory arrays, PIM cannot integrate large, power-hungry, and complex processing elements without compromising scalability and efficiency. As a result, simply scaling up PIM compute capability is neither practical nor sustainable. Instead, lightweight and efficient computation mechanisms, such as CCA, are required to alleviate the computational bottlenecks within PIM architectures.

The hot code-centric feature of the PIM applications also supports these findings. Figure 4 shows the primary computations on hot code regions, and the proportion of time for each workload in Group 1. As shown in the figure, most benchmarks spend a significant number of cycles for hot codes, up to 95%. Since the computational bottleneck in PIM is caused by the limited compute capability of the PIM processor, the hot codes can be categorized into key operations such as multiply-and-add and accumulation. Thus, introducing a single flexible accelerator with a simple structure for these operations can effectively achieve both high performance and wide coverage. These findings suggest that CCA is well-suited to PIM, and can also meet its limited area and power constraints.

Benchmark	Hot Codes
GEMV	Multiply-and-Add Reduction
MLP	Multiply-and-Add Reduction
TRNS	Index Processing
TS	Multiply-and-Add
GEMM	Multiply-and-Add Reduction

(a)



(b)

Figure 4: (a) The core operations (hot codes) of compute-intensive PIM benchmarks and (b) the percentage of execution cycles.

2.4 Hardware Constraints & Opportunity for Applying CCA in PIM Architecture

PIM architectures face inherent hardware constraints due to fundamental differences in memory and processor fabrication. Since PIM integrates memory and computational units on a single chip, it inherits the limitations of memory-centric manufacturing. For example, memory chips typically utilize only three metal layers, whereas modern processors use over ten layers to support dense, high-speed interconnects. This disparity reduces the number of possible parallel data paths and increases signal resistance, ultimately constraining bandwidth and computational throughput. Additionally, the co-location of memory and compute units introduces resource contention, necessitating sophisticated arbitration mechanisms that further complicate the design. Thus, as discussed in Section 2.3, scaling PIM with complex, power-intensive processing elements is infeasible under these memory-imposed constraints.

Overcoming these limitations requires specialized strategies that minimize data movement and optimize memory-compute interactions. Potential solutions include hierarchical or vertical interconnect schemes, reduced instruction sets tailored for in-memory operations, and adaptive cache or prefetching mechanisms. However, once memory bottlenecks have been mitigated, the architecture must address the emergence of computational bottlenecks. In this context, CCA can be a promising solution. Its modular, low-overhead design is well-suited to the limited wiring resources of memory systems, enabling target performance improvements without compromising the core advantages of PIM. Therefore, bridging the gap between memory constraints and increasing computational bottlenecks is important and challenging. This can be effectively addressed by CCA-enabled PIM architectures, which tightly integrate CCA and PIM based on a deep understanding of PIM workload characteristics.

2.5 Balancing Tasklet Parallelism for CCA-Enabled PIM Design

While PIM reduces data communication costs by performing computations closer to memory, it is still constrained by the requirement to retrieve data from memory banks or cells. As a result, these memory load/store instructions cannot be completed in a single cycle and, although memory stalls have shorter latency than those of traditional processors, they still persist. As discussed in Section 2.1, the baseline UPMEM architecture requires at least 11 tasklets per DPU to fully utilize its pipeline stages and hide instruction dispatch latency. Although more tasklets can mitigate latency and reduce control and data hazards in hot codes, they are less effective in memory-bound regions. Therefore, tasklets frequently stall while awaiting memory fetches, thereby preventing full latency hiding. Thus, performance tends to saturate beyond a certain tasklet count. Moreover, increasing the number of tasklets introduces control and scheduling overheads. Therefore, for memory-intensive code blocks, using a moderate number of tasklets is more efficient than attempting full DPU utilization with the maximum number of tasklets.

In existing DPUs, the number of tasklets must be statically defined and uniformly applied across all code regions within a workload, regardless of their computational or memory characteristics. As a result, developers often increase the number of tasklets to overcome computational bottlenecks, despite the associated scheduling and synchronization overheads. However, a CCA-enabled PIM design can alleviate these bottlenecks by offloading compute-intensive tasks to the CCA. This allows high performance to be achieved even with fewer tasklets, as both compute- and memory-bound regions can be efficiently processed. Nevertheless, using too small number of tasklets may result in under-utilization of both the DPU and the CCA, leading to suboptimal performance. Therefore, balancing the degree of tasklet parallelism based on workload characteristics is essential for maximizing memory bandwidth and computational efficiency. CCA-enabled PIM designs offer opportunities to improve overall system throughput and efficiency by enabling more flexible, workload-aware resource utilization.

2.6 Summary and Insights

In summary, our analysis reveals three key insights that could be used to improve PIM architectures. First, while current PIM designs effectively alleviate the memory wall, this often shifts the performance bottleneck towards computation. Second, due to the tight integration of processors within memory chips, scaling up computational capability is constrained by memory-specific physical limitations. Third, although tasklets help to hide operation latency by utilizing pipeline stages efficiently, they introduce scheduling overhead and exhibit diminishing returns for memory-bound workloads. These observations motivate a CCA-enabled PIM design that integrates lightweight, compiler-guided CFUs that target hot code regions, as well as fine-tuning of tasklet counts based on workload characteristics. By offloading critical subgraphs to the CCA, the proposed design can (1) support high arithmetic throughput without increasing the complexity of PIM processors, (2) retain the area and power constraints of memory fabrication, and (3) reduce tasklet management overhead while maintaining high performance. This co-design enables more balanced utilization of memory bandwidth and compute resources, ultimately delivering improved system-level throughput and energy efficiency compared to conventional PIM or standalone accelerator architectures.

3 PIM-CCA: A Novel CCA-Enabled PIM Architecture

Motivated by the aforementioned insights, we propose ***PIM-CCA***, a novel CCA-enabled PIM design intended to alleviate computational bottlenecks in existing and future PIM architectures. Figure 5 shows a high-level overview of PIM-CCA. We first analyze hot codes in existing PIM workloads through application analysis and identify the main performance bottlenecks that can be accelerated using CCA. Based on these insights, we introduce PIM-aware CFU and CCA design strategies that effectively enhance the computational capabilities of PIM processors while meeting memory-level constraints. To ensure compatibility with existing PIM architectures, we implement a CCA-enabled PIM system based on the UPMEM architecture. Furthermore, we optimize tasklet management in the CCA-enabled PIM architecture to efficiently exploit both memory- and compute-intensive workloads.

While the current PIM-CCA is designed for UPMEM PIM architectures and implemented on the uPIMulator [16], the CCA approach offers high design flexibility and adaptability, enabling seamless integration with various processor architectures [6, 7]. Therefore, the strategy of applying CCA to PIM is not limited to UPMEM; it can be applied to other PIM architectures to effectively mitigate computational bottlenecks in PIM workloads.

3.1 Hot Code Detection and Analysis

To identify the key operations for configuring CFUs and accelerating them with CCA, we first analyze PIM workloads. The primary concept of hot code detection involves three steps: finding promising hot code candidates at the dataflow level, verifying them in a cycle-accurate simulator, and finally selecting subgraphs to offload to CCA considering hardware conditions. Specifically, we use LLVM's front-end compiler to compile the application into IR and identify common patterns that appear in the innermost loops of IR-level dataflow graphs or across multiple applications. These include multiply-add, accumulation, complex index calculations, and others. We also verify that the discovered patterns account for a significant proportion of total execution cycles using a cycle-accurate PIM simulator (uPIMulator). Finally, we determine whether the verified patterns meet the hardware requirements and are suitable for offloading. For example, offloading a multiply-add pattern to CCA is generally undesirable in PIM because processing the entire multiplication operation with CCA requires a significant amount of hardware and power. Instead, it makes more sense to offload part of the multiplication operation to a simple CCA. If there are sufficient resources available, using a larger CCA for the multiply-add operation could be beneficial.

By recursively detecting hot code regions across diverse PIM workloads, we identify key operations with high potential for performance gains when accelerated via CCA. Our analysis of PrIM benchmark workloads on the UPMEM system reveals that multiplication and reductions constitute the most performance-critical

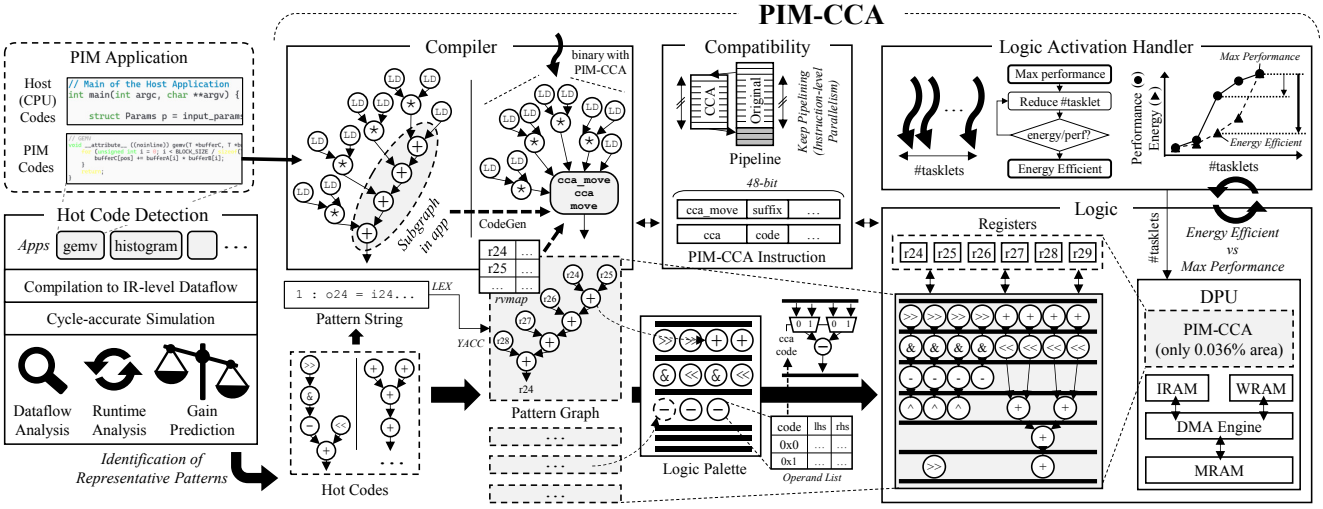


Figure 5: High-level overview of PIM-CCA.

hot code regions. We use these identified hot codes as the basis for designing the CCA logic, which is discussed in Section 3.3.4

3.2 PIM-Aware CFU and CCA Design Strategies

Based on the results in Section 3.1, we determine how to construct CFUs and the CCA. The CCA's objective is to accelerate hot code execution by offloading the region from the main PIM processor. By designing the CCA to handle the operations identified in the hot code region, the PIM-CCA architecture can significantly reduce the number of cycles and improve overall system performance.

The design of the CCA incorporates CFUs that correspond to computational patterns identified through hot code analysis. These CFUs are integrated into the existing processor, extending its instruction set so that hot code operations can be offloaded directly. This integration allows the PIM to support both general-purpose and highly optimized application-specific tasks without sacrificing flexibility. However, the hardware overhead of CFU integration must be considered carefully, particularly given the strict area and routing constraints inherent to PIM architectures, as discussed in Section 2.4.

Therefore, we propose PIM-aware CFU design strategies, including unrolling intrinsic macros into simpler sequences of instructions at assembly level. Our analysis reveals that many workloads are compute-bound by multiplication operations, which are inefficiently handled by the baseline system. For example, multiplication on UPMEM is implemented as a series of tiny multiplication-step (mul_step) instructions spanning 32 cycles for 32-bit integers, which highlights the limited computational capability of the embedded processing units. Notably, these mul_step sequences are composed of a small set of recurring instructions across a wide range of workloads. This observation suggests that consolidating multiple mul_step operations into a single CFU could provide significant performance benefits across diverse workloads.

Therefore, the number of mul_steps is set to a value that does not exceed the pipeline depth of the baseline architecture. This maintains compatibility with the existing design and adheres to area constraints. Figure 6 shows the multiplication throughput for 2^{16} integers across different CCA sizes, focusing on mul_steps. The speedup tends to saturate at 4 mul_steps, while the number of execution stages increases logarithmically due to the accumulation required in the final stage. Configuring more than 4 mul_steps exceeds the allowable pipeline depth, resulting in pipeline stalls and additional control complexity.

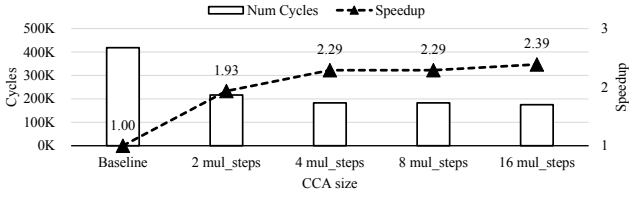


Figure 6: Multiplication throughput for various CCA sizes.

Based on these insights, we select the appropriate hot codes and corresponding sizes, and design the CFU accordingly. By tailoring the CFU design to these commonly used instruction subsets, and conforming to the area and wiring constraints of PIM, we develop a lightweight yet effective CCA architecture optimized for integration into the UPMEM DPU. This strategy carefully balances the trade-offs between computational performance, silicon footprint, and routing complexity, making it well-suited for memory-centric systems with strict hardware limitations.

3.3 Construction of CCA-Enabled PIM Architecture

In this section, we present the detailed design of the CCA-enabled PIM architecture, based on the strategy outlined in Section 3.2.

3.3.1 CCA Logics. We designed the CCA logic based on the identified hot codes, and the detailed design approach is discussed in Section 3.3.4. The CCA logic consists of four right shifters, four

adders, four subtractors, four left shifters, and a two-level accumulator. The accumulator consists of two adders on the first level and another two adders on the second level, enabling reduction-style computations, as shown in Figure 7. In the CCA, the mul_step, the most critical execution sequence, is decomposed into a 7-stage pipeline (from CCA Execution 0 to CCA Execution 6 in Figure 7) that sustains high throughput while supporting thread-level interleaving. The datapath is partitioned into two halves, each of which is responsible for half of the operations. The first stage of CCA execution issues four right-shift units on the left and four adders on the right. Stage 2 applies four bitwise-AND units (left) and four left-shift units (right). Stage 3 uses four subtractors on the left to generate predicate signals that determine whether the right-shifted values should be accumulated. These predicates then control a conditional-routing network in Stage 4, which forwards only the relevant intermediate results to Stage 5. Stages 5 and 6 use a cascade of two adders followed by a single adder to compute partial sums. Stage 7 finalizes the accumulation and returns the result to the main pipeline.

Since the logic can be reconfigured at runtime, the datapath supports a wide range of sub-execution flows. The right-side shift-adder cluster handles general accumulation and partial-sum kernels, while the left-side subtractor cluster supports three-operand max selection via conditional subtraction and comparison. The compiler identifies these patterns and maps each one to one of the 2^7 entries in the CCA code cache by emitting a 7-bit CCA code, enabling a rich set of execution flows without hardware duplication. As a result, the CCA achieves both functional versatility and high reuse efficiency within a fixed 9-stage pipeline, including decode and return stages, meeting the stringent area and energy constraints of PIM architectures.

As shown in Figure 7, all CFUs are interconnected via a lightweight multiplexer (MUX). Directly managing the connections for each operation would be complex and inefficient. To address this, we construct a CCA lookup table which specifies the detailed MUX configurations required for each supported operation. By mapping CCA control codes to MUX settings, we maintain a unified datapath between the CCA and the DPU, thereby eliminating operand-decode overhead and maximizing code density. This mechanism enables flexible, multi-stage operations to be executed from a single CFU invocation.

The CCA lookup table is generated at compile time based on identified hot code regions and their corresponding CFU execution flows. Once constructed, the table persists across subsequent compilations, enabling new workloads to reuse or extend existing mappings between CCA codes and control flows. This allows the table to evolve over time, supporting a wider range of computational patterns without requiring hardware modifications. At runtime, the program performs a lightweight table lookup using the CCA control code to retrieve the appropriate MUX settings for CFU configuration. This enables fast and deterministic CCA configuration and hot code acceleration, while maintaining the programmability of the baseline architecture.

Finally, the CCA-dedicated register file is connected to the main processor's register file. Operands are conventionally fetched from the main register file, and then written into the CCA register file in the same order. This allows the CCA register file to be designed

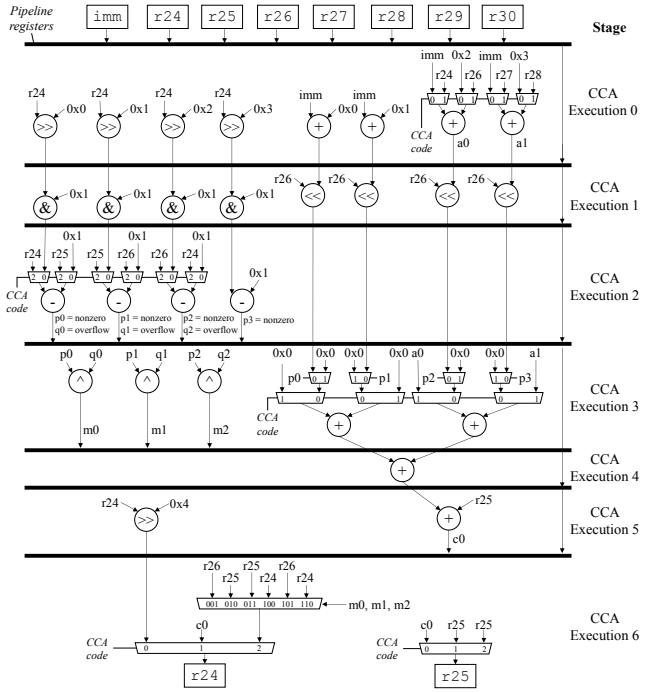


Figure 7: The PIM-CCA logic structure.

simply without requiring major modifications to the main register file. To support parallel access, the CCA register file provides multiple read/write ports based on operand requirements, thereby minimizing execution stalls.

3.3.2 ISA Extension and Pipeline Compatibility. The baseline UPMEM system uses a 48-bit instruction word. The upper 6 bits identify the opcode, the next 7 bits encode the opcode-related metadata, and the remaining bits are reserved for operands. The UPMEM DPU employs a 14-stage pipeline, with only the final three stages (ALU4, MERGE1, MERGE2) overlapping with instruction fetch and dispatch. Therefore, to avoid pipeline bubbles, CCA execution must be completed and deliver the result by the final CCA Return stage. To satisfy this constraint, we have extended the baseline instruction set architecture (ISA) with two custom instructions: *cca* and *cca_move*. The instruction words are designed to be compatible with the baseline system while avoiding the need for additional decoding units (Figure 8). The size of the CCA lookup table is determined by the decoding scheme of the target architecture, which is 2^7 in UPMEM.

We connect the CCA datapath to the FETCH1 stage of the DPU, enabling immediate branching for *cca* instructions. Figure 9 indicates the CCA pipelines. Following the ISA format, we use the upper 6 bits to identify *cca* instructions alongside native DPU instructions, and the next 7 bits to encode the CCA code. As UPMEM instructions are fetched in three stages, each decoding 16 bits, we position the CCA opcode and the CCA code in the upper 16 bits to allow early detection and offloading in the initial fetch stage. The CCA code determines the CFU configuration in the CCA. We also introduce a separate instruction, *cca_move*, to cover the read operand

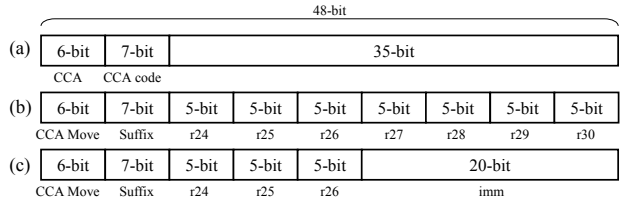


Figure 8: The PIM-CCA instruction formats. (a) *cca* and (b), (c) *cca_move*.

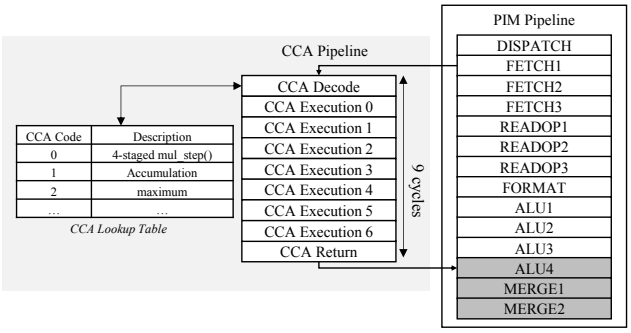


Figure 9: The PIM-CCA pipelines.

stage. *cca_move* is executed before the *cca* instruction to preload the CCA registers with values. As there is a limited number of CCA registers, and the CCA is executed in the inter-tasklet pipelining region of UPMEM, as shown in Figure 9, the cca_move instruction can be executed without a separate hazard or pipeline stage. To enable direct data transfer for cca_move, we introduce additional hardware ports that connect the DPU register file to the CCA-side register bank. This allows efficient data transmission to the CCA without stalling or disrupting the DPU pipeline. In our design, a single cca_move instruction can transfer up to six operands, which may be a register value, an immediate constant, or an address label, in a single execution. This design minimizes disruption to the DPU pipeline and maintains compatibility with the baseline architecture.

To maintain compatibility with the DPU pipeline, the CCA is carefully pipelined to match the execution latency of the remaining DPU stages. Since the DPU requires 8 to 9 cycles after a CCA branch, the CCA is implemented as a 9-stage pipelined accelerator. This ensures that instructions executed in the CCA return to the DPU's MERGE stage with precise timing, aligning exactly with the original pipeline's latency window. This design preserves pipeline balance, eliminates bubbles, and enables the efficient integration of custom code acceleration within a tightly constrained execution model.

3.3.3 PIM-CCA Compiler. To preserve programmability and flexibility, we introduce the PIM-CCA compiler, which identifies and replaces hot code regions with CCA instructions based on DFG analysis. Figure 10 shows (a) an example of a hot code pattern string and (b) its corresponding graph representation. A pattern string consists of CCA code and its associated assignment statements. The left-hand side of each statement represents an output or temporary register, while the right-hand side is a mathematical expression

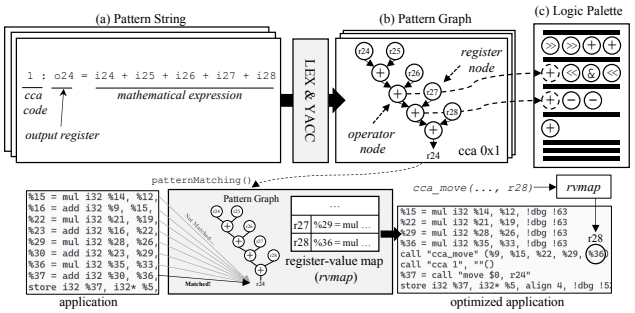


Figure 10: A pattern string-based PIM-CCA compiler. (a) Pattern string, (b) pattern graph and (c) the logic palette.

Algorithm 1 Pattern matching algorithm

Require: Value *v*, Node *n*, Map *rvmap*

```
1:  if isOperatorNode(n) then
2:      if v is not instruction then return false end if
3:      I ← cast<instruction>(v)
4:      if I.opcode is not n.opcode then return false end if
5:      matched ← and(for_each(patternMatching,
        {I.operands, n.childs, rvmap}))
6:      if not matched and isBidirectionalNode(n) then
7:          rvmap.restore()
8:          matched ← and(for_each(patternMatching,
            {I.operands, reversed(n.childs), rvmap}))
9:      end if
10:     return matched
11: else if isRegisterNode(n) then
12:     if rvmap.find(n) then
13:         return rvmap.at(n) == v
14:     else
15:         rvmap.insert({n, v})
16:         if n.linkedSubgraph then
            return patternMatching(v, n.linkedSubgraph, rvmap) end if
17:         return true
18:     end if
19: end if
```

composed of input and temporary registers, representing a hot code and its associated CFUs. The compiler parses the pattern string using Lex and Yacc, to construct a corresponding pattern graph. This comprises nodes representing the operations and registers involved in the expression. Each graph corresponds to a single assignment statement, with a root at the left-hand side register. To enable efficient matching, multiple pattern graphs are merged through shared temporary or output registers. The compiler then compares the constructed pattern graph with the target instruction stream to identify code blocks suitable for replacement with CCA instructions.

Algorithm 1 shows the process of matching the pattern graph against the target instructions. For operator nodes, the compiler checks whether the node's operation matches the opcode of the target instruction (line 4). It then recursively verifies the child nodes against the operands of the instruction (line 5). For commutative (bidirectional) operators, the compiler allows operand matching in reverse order to increase matching flexibility (lines 6–9). If all

Algorithm 2 Logic palette construction algorithm

```
Require: Logic Palette P, Pattern Graph G
1:  while minimizing the cost do
2:      cost := 0
3:      while All the nodes n in G are matched to P do
4:          if n.parent not matched then continue end if
5:          for Stage s in P after stage matched with n.parent do
6:              if there is computing unit cu which is not reserved and
                matched with n then
7:                  cu.reserved = n
8:              else
9:                  add new computing unit to P with n
10:                 cost += 1
11:             end if
12:         end for
13:     end while
14:     evaluate result with cost
15:     reshape P
16: end while
```

comparisons succeed, the compiler determines that the target instruction can be replaced with equivalent CCA instructions (line 10). For register nodes, the compiler uses a mapping structure, called *rvmap*, which associates pattern graph registers with concrete values from the application. If a value is already mapped to a register node (line 12), the compiler checks whether it matches the current operand (line 13). If no mapping exists, the compiler assigns the register node to the current value (line 15) and continues with the remaining comparisons (lines 16–17). After pattern matching, the PIM-CCA compiler replaces the identified code blocks with the relevant CCA instructions.

3.3.4 CCA Logic Construction. Based on the analysis in Section 3.1, we design the CCA logic using hot code regions that are represented by pattern strings and graphs. To facilitate this process, we introduce a logic palette. Figure 11 (a) illustrates the organization of the logic palette, which consists of multiple stages, each containing several computing units. Both the stages and computing units directly correspond to the actual pipelined CCA logic. Each computing unit maintains an operand list based on the CCA code and is implemented using a MUX, as shown in Figure 11 (a). As shown in Figure 10 (c), the logic palette is constructed from multiple pattern graphs. The primary goal is to generate the final CCA logic while minimizing hardware costs by maximizing the overlap of operations across pattern graphs. Since maximizing such overlap is an NP-hard problem, we use a heuristic approach to efficiently integrate multiple pattern graphs into the logic palette.

Figure 11 (b) and Algorithm 2 illustrate the construction process of the logic palette using multiple pattern graphs. As shown in Figure 11, each node in the pattern graphs can be integrated into the logic palette using one of three methods: (b-1) matching an existing computing unit, (b-2) adding a new computing unit, or (b-3) inserting a new stage. These methods are applied to stages that follow those matched to parent nodes. Algorithm 2 shows how to construct the palette to minimize logic cost as much as possible (line 1). The process continues iteratively until all nodes have been integrated using one of the three methods (line 3). In addition to verifying the parent node condition (line 5), integration ensures that computing units are not preempted by other nodes (line 6). Method (b-1) is prioritized due to its minimal overhead, requiring only a MUX (lines 6-7), whereas method (b-2) incurs a higher hardware cost by adding new computing units (lines 9-10). Method (b-3), which adds a new stage, is selectively applied by reshaping the palette (line 15). Once all nodes have been integrated, the overall cost is evaluated (line 14), and the process repeats until the minimum logic cost is achieved (line 1).

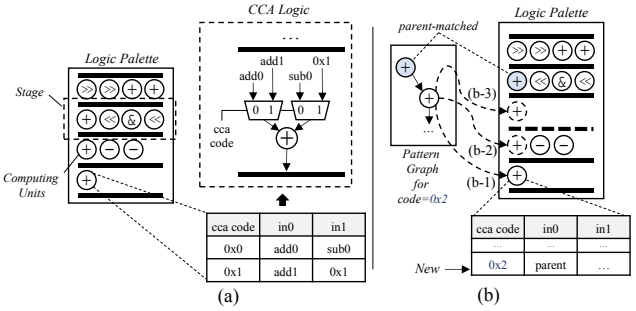


Figure 11: (a) The logic palette and its usage to build the CCA logic. (b) The logic palette construction using pattern graph.

3.4 Balancing the Logic Activation

As discussed in Section 2.5, memory stalls can still occur in PIM systems. Consequently, fully utilizing all pipeline stages of DPUs does not necessarily improve performance, particularly for memory-intensive instructions. Although reducing the number of tasklets does not significantly improve overall performance, it helps to reduce scheduling and control overhead. Since CCA mitigates computational bottlenecks within the DPU, a CCA-enabled PIM architecture can achieve comparable or even superior performance to the baseline by using fewer tasklets in both memory- and compute-intensive regions. For example, in the proposed CCA architecture based on UPMEM, up to four mul_step instructions can be executed concurrently, reducing the total instruction count by approximately 75%. As a result, for workloads where CCA fully eliminates computational bottlenecks, performance tends to saturate before all pipeline stages are utilized, allowing optimal performance with fewer tasklets. However, for workloads where bottlenecks are only partially resolved, increasing both DPU and CCA utilization continues to improve performance.

Reducing tasklet management overhead usually results in modest gains, of approximately 1–2 cycles per tasklet on each PIM unit. Moreover, as tasklet scheduling often occurs simultaneously with execution, its direct impact on performance is limited. However, as modern PIM systems scale up to include thousands of PIM units, minimizing such overhead becomes increasingly important in terms of energy efficiency. In cases where the target workloads exhibit low intensity, simplifying the tasklet management logic itself may be beneficial. From this perspective, identifying the appropriate number of tasklets for each workload allows for cost-effective execution, even without full processor utilization. However, because the optimal number of tasklets varies depending on the workload characteristics, especially within a CCA-enabled PIM system, it is essential to carefully tune this parameter to maximize efficiency.

Table 1: The PIM-CCA instructions. Each i-, t-, and o-register in pattern string representing the input, temporary, and output registers, respectively.

CCA code	Description	Pattern String
0x0[1]	4 step of multiply	o25 = i25 + mul_step(i24, i26, imm); /* repeat 4 times */
0x1	accumulation	o24 = i24 + i25 + i26 + i27 + i28
0x2	max	t0 = i24 >i25 ? i24 : i25; o24 = t0 >i26 ? t0 : i26

[1]we manually fix codes for this CCA operation because UPMEM SDK implement multiply using inline assembly with mul_step instruction in syslib.

We propose a compile-time mode selection mechanism that allows selecting either maximum-performance or energy-efficient mode. In the energy-efficient mode, the framework determines the tasklet count via straightforward yet effective iterative exploration.

4 Evaluation

4.1 Methodology

Our benchmarks include six compute-intensive workloads for which performance can be improved by offloading complex computations to CCA, and seven memory-intensive workloads which number of tasklets can be adjusted by observing control overheads based on the PrIM benchmark suite [13]. Some PrIM benchmarks (BFS, NW and SpMV) are excluded from our evaluation due to limitations in the instruction set supported by our baseline simulator. We include General Matrix-Multiplication in our evaluation as a compute-intensive workload for which a significant number of instructions can be offloaded to CCA. The input data for the PrIM benchmarks are set to their default values, while GEMM uses input matrices of size (32,128) x (128,128).

We evaluated the effectiveness of PIM-CCA using a cycle-level PIM simulator [16], which models the UPMEM PIM architecture. We used the same core frequency as the UPMEM DIMM (350MHz) in our PIM-CCA environment, and we evaluated in a single-bank condition to test the computational improvements of a single core.

We conducted experiments in both single- and multi-DPU environments, and the results showed no noticeable performance differences, as expected. This is because overall performance in a multi-DPU environment is only affected by inter-bank communication, but we assume that each DPU is equipped with its own CCA and our benchmarks do not involve inter-bank communication. We used flex 2.6.4, bison 3.5.1 [27] and LLVM 12.0.0 [24] to build the PIM-CCA compiler.

As shown in Table 1, we use three CCA operations derived from the PIM-CCA compiler. We set our base CCA operation to *four steps of multiplication* represented as CCA code 0x0. From this base operation, the PIM-CCA compiler the found *accumulation*, represented by CCA code 0x1, which can be used in the GEMV, GEMM, Multi-Layer Perceptron (MLP) and Reduction (RED) benchmarks. We also identified *max* operation, represented as CCA code 0x2, in Needle Wushiman (NW) in PrIM benchmarks, while NW is unavailable in the UPMEM PIM simulator environment.

To comprehensively assess the computational improvements and compiler-level contributions of PIM-CCA, we conducted experiments focusing on key aspects such as execution performance, computational intensity mitigation, control overhead, and hardware overhead.

4.2 Overall Performance

Figure 12 (a) shows the relative cycle-level performance (Y-axis) and Figure 12 (b) shows the total number of executed instructions (Y-axis), comparing PIM-CCA with the baseline in various benchmarks (X-axis). In each graph, the cycle-level performance on the Y-axis is normalized to that of the baseline UPMEM DIMM with 16 tasklets.

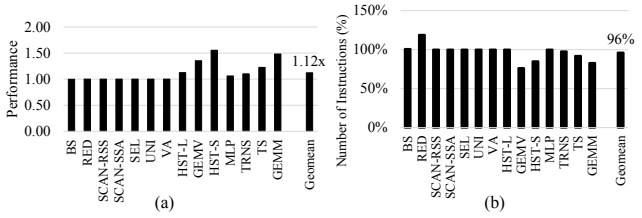


Figure 12: Comparisons of (a) relative performance and (b) number of instructions of PIM-CCA over baseline with 16 tasklets.

In compute-intensive workloads, the PIM-CCA achieves performance improvements ranging from 1.06x to 1.55x, with a geometric mean of approximately 1.26x over the baseline. Notably, workloads such as GEMV and GEMM exhibit substantial performance gains due to their high arithmetic intensity and extensive use of multiplications, which are well-accelerated by the CCA. In contrast, the MLP workload, despite its similar computational structure, shows comparatively modest improvement. This discrepancy arises because the input data in MLP is relatively small, allowing the baseline system to swiftly complete the multiplication phase, even without CCA acceleration. As a result, MLP makes limited use of the mul_step sequence, which is a major performance bottleneck in other workloads. The accumulation, represented by CCA code 0x1 in Table 1, accounts for up to 19.7% of the total performance improvement in the GEMM benchmark.

The trend in the number of executed instructions generally aligns with the observed performance improvements, though it does not exhibit a strong correlation. This is primarily due to the presence of cca_move instructions, which are required to transfer data from the Register File to the CCA pipeline prior to execution.

In RED, the instruction count increases with CCA, while performance remains similar to the baseline. This is due to the insertion of repeated cca_move operations accompanied by cca operations, as well as the optimization behavior of the existing compiler with respect to neighboring instructions. Since the RED benchmark is composed of many accumulations (CCA code 0x1) that use only a subset of the CCA logic, the number of inserted cca and cca_move instructions can be equal to the number of removed instructions. This is different from other multiplication-dependent benchmarks and is the reason of the limited performance gain.

On the other hand, memory-intensive workloads do not exhibit noticeable performance improvements compared to the baseline. This is primarily because such workloads contain minimal computational operations, unlike compute-intensive applications. The computations in these workloads are typically lightweight operations, such as additions or subtractions, which are not offloaded to the CCA due to their limited complexity and low latency.

Even when small subsets of instructions can be offloaded to the CCA, the overhead of additional cca_move operations for operand preparation often outweighs the potential performance gains. As discussed earlier, this overhead is non-negligible, especially when the computational complexity is insufficient to amortize the cost of operand transfers. As a result, for memory-bound workloads with minimal or trivial computations, utilizing CCA does not yield measurable performance gains and may introduce unnecessary execution overhead.

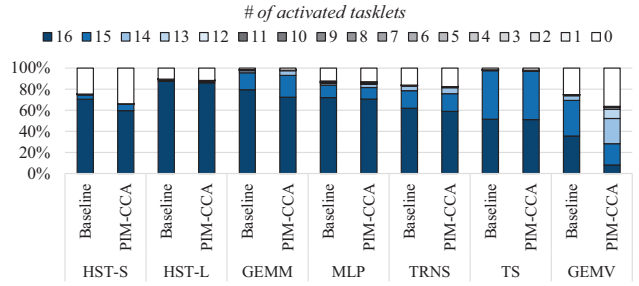


Figure 13: Comparison of compute intensity across compute-intensive workloads with varying tasklet counts.

4.3 Compute-Intensity Mitigation

To conduct a more detailed analysis of compute-intensive workloads, we examine the distribution of execution cycles with varying numbers of activated tasklets. Figure 13 shows the tasklet activation rates (Y-axis) compared to the baseline system in compute-intensive workloads (X-axis). The Y-axis represents the percentage of the cumulative number of tasklets activated at each cycle throughout the entire execution. We define compute-intensity as the percentage of cycles utilizing 11-16 tasklets, indicating the degree to which the available parallelism is exploited in the PIM architecture. Our results show a reduction in compute-intensity of up to 11.62% (GEMV). This indicates that PIM-CCA effectively alleviates computational pressure by offloading heavy operations to the CCA, thereby reducing reliance on high tasklet parallelism. Therefore, our design demonstrates the ability to transform compute-intensive behavior into more PIM-friendly execution patterns, aligning well with the characteristics of PIM architectures.

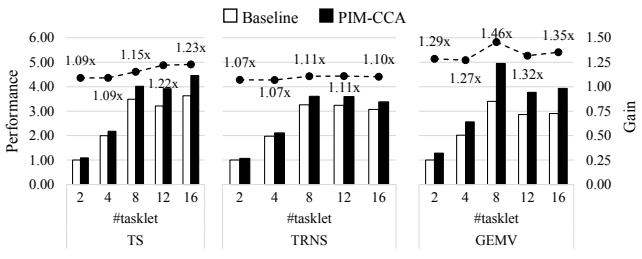


Figure 14: Comparisons of relative performance and gains of PIM-CCA in compute-intensive workloads.

4.4 Performance Gain in Compute-Intensive Workloads

To determin the effect of CCA across different compute-intensity types, we analyze the relative performance gains achieved by offloading key computations to CCA in compute-intensive workloads. The results reveal three distinct trends across different workload types. Figure 14 compares the performance of PIM-CCA (left Y-axis) with the performance gain (right Y-axis) for three key compute-intensive workloads with various numbers of tasklets (X-axis). For each workload, the left Y-axis shows performance normalized to the baseline using 2 tasklets, while the right Y-axis presents the performance gain of PIM-CCA over the baseline for each tasklet configuration.

TS, which has the highest compute intensity, shows a consistent increase in performance when the number of tasklets increases. In these cases, both raw performance and performance gain over the baseline increase with higher tasklet parallelism. This trend highlights the presence of remaining computation bottlenecks within the PIM architecture. As a result, for highly compute-intensive workloads, maximizing tasklet utilization remains beneficial even with CCA integration.

TRNS, which belongs to the mid-level compute intensity category, shows saturation in both performance and gains at around 8 tasklets. This indicates that increasing the tasklet count beyond this point will not yield significant additional benefits. Notably, with CCA, the number of activated tasklets can be reduced without sacrificing performance for moderately compute-intensive workloads. The primary computational bottleneck in TRNS is address calculation, which involves multiple multiplications that are not MAC operations, unlike TS, MLP, GEMM, and GEMV. This result shows that PIM-CCA can improve not only MAC-dominant workloads, but also those with other architecture-specific bottlenecks.

GEMV, which has a lower compute intensity, achieves peak performance at a tasklet count below the maximum supported (i.e., 11 tasklets, which fully utilize the UPMEM pipeline). In these near-memory-intensive scenarios, both performance and performance gains are maximized with fewer tasklets than the maximum number supported. This indicates that excessive tasklet parallelism could introduce a performance overhead that outweighs the benefits.

These trends collectively demonstrate that the optimal tasklet configuration depends heavily on workload characteristics, and that CCA-enabled systems can benefit further from adaptive thread allocation strategies that consider computational intensity to balance performance and overhead.

4.5 Control Overheads with Reducing Tasklets

Figure 15 shows control overhead (left Y-axis) and relative performance (right Y-axis) with increasing tasklet counts (X-axis). Control overhead is quantified by observing the behavior of the tasklet scheduler, which incurs additional scheduling overhead to find the next available (executable) tasklet in situations where the majority are stalled due to memory accesses. This overhead becomes increasingly noticeable in memory-bound scenarios, where frequent stalls limit the available parallelism. The performance on the right Y-axis is normalized to that of 16 tasklet-based execution.

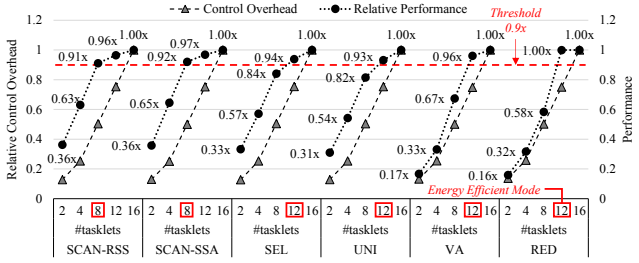


Figure 15: Comparison of relative performance and control overhead of PIM-CCA in memory-intensive workloads.

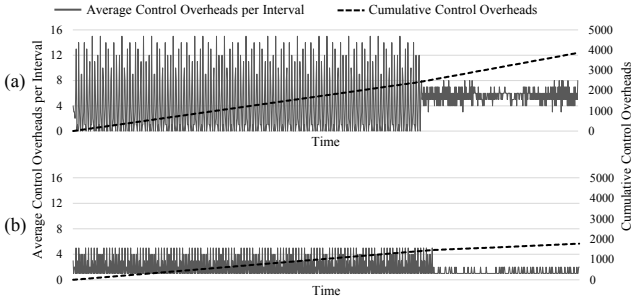


Figure 16: Change of control overhead over time during the execution of SCAN-SSA with (a) 16 tasklets and (b) 8 tasklets.

For memory-intensive workloads, we evaluate the impact of scheduling and control overhead on energy efficiency. As shown in Figure 15, the performance saturation point varies across different workloads. However, they all exhibit a similar trend of increasing control overhead as the number of activated tasklets increases.

In our evaluation, RED and VA, showing the highest memory-intensity in Figure 3, reach performance saturation at around 12 tasklets, corresponding to full pipeline utilization. In contrast, SCAN-RSS and SCAN-SSA, showing the lowest memory-intensity, exhibit less than 10% performance degradation when the tasklet count is reduced to 8, while achieving approximately 50% reduction in control overhead. This demonstrates that, for some memory-intensive workloads, limiting tasklet parallelism can lead to significant energy and control efficiency gains with minimal performance loss. Workloads such as SEL and UNI fall between these two extremes, showing moderate saturation behavior and providing further evidence that tuning the number of tasklets based on workload characteristics is a viable strategy for optimizing PIM execution.

Figure 16 shows the average control overhead (left Y-axis) per interval and the cumulative control overhead (right Y-axis) over time (X-axis) for the SCAN-SSA benchmark. Control overhead is averaged over intervals of 10k consecutive cycles. As shown in the figure, reducing the number of activated tasklets significantly lowers the control overhead during execution. As a result, the total cumulative control overhead decreases by approximately 54%, while performance degrades by only about 8%.

4.6 Hardware Overhead

We evaluated the additional hardware overhead introduced by PIM-CCA, which integrates a lightweight arithmetic functional unit into the PIM device. The design was implemented in Verilog HDL as a 7 execution stages, capable of performing 4-staged mul_steps, accumulation, or maximum operations depending on the cca_code. Synthesis was performed using Synopsys Design Compiler [39] targeting a 28nm CMOS technology, and the results were the scaled analytically to a 25nm process to match the fabrication node of UPMEM PIM device [43].

To enable a fair comparison under consistent technology assumptions, we applied a simplified scaling model based on the Dennard scaling principle [8]. Assuming a constant supply voltage, the area was scaled quadratically, and the dynamic power was scaled linearly with respect to the feature size. The resulting area overhead is 4464.29 μm^2 based on a 4-metal-layer P&R implementation, corresponding to only 0.036% of the total area of an UPMEM chip [9]. The corresponding power overhead is 1.28 mW, accounting for only 0.73% to 1.46% of the UPMEM's total power consumption depending on operating conditions. These results demonstrate that PIM-CCA introduces negligible hardware overhead while enabling additional computational capabilities within the PIM architecture.

To preserve the pipeline structure of existing PIM devices and minimize hardware overheads such as area and power consumption, the PIM-CCA design is intentionally constrained to a minimal logic footprint. While these constraints promote lightweight integration, they can also provide flexibility for future enhancements. In this case, we can add a div_step operation, which is less frequent than the mul_step operation, yet still prevalent in many workloads.

5 Discussion

In this paper, UPMEM is used as the reference architecture; however, the proposed CCA-enabled design is not limited to UPMEM. CCA is a deeply pipelined on-chip array whose routing fabric and the mix of functional units are configurable. Enhancing the main PIM processor may appear to be the most straightforward approach to improving computational capability. However, hot codes vary across workloads and are often intermittent and interleaved with memory operations. Moreover, as discussed in Section 2.3 and 2.4, PIM architectures face strict area and power constraints due to their memory-centric nature. Thus, enhancing the PIM processor directly is often inefficient and leads to increased HW costs. In contrast, CCA can effectively complement the limited functionality of PIM by leveraging the advantages of reconfigurable designs such as CGRA, FPGA, and systolic arrays.

Architectural Coverage PIM exhibits diverse implementations depending on the underlying memory type((G)DDR or HBM) and the placement of DPUs [14, 19, 21, 23, 25, 26]. UPMEM [9, 43, 44] and GDDR6-AiM [23] integrate DPUs at each bank, whereas HBM2-PIM [19] employs DPUs connected across multiple banks. The CCA is designed to accelerate computations directly alongside the PIM processor and can be integrated into each DPU in both types. Furthermore, in HBM-based designs, CCA can also be integrated into the logic or buffer die, in addition to being deployed at each DPU.

CCA can achieve its maximum potential when combined with a programmable, pipelined PIM processor. However, command-driven PIM designs with fixed-function support can also benefit

from CCA. These PIM architectures are positioned closer to memory and employ simpler DPUs that support only a limited set of operations. By enabling a broader range of operations, CCA offers greater architectural flexibility and allows a wider variety of workloads to be executed directly in memory. This enhances data reuse, reduces off-chip communication, and ultimately improves both performance and energy efficiency.

While external accelerators with higher FLOPs offer better computational throughput, they often incur substantial off-chip communications and require additional peripherals such as controllers. Moreover, direct access to in-PIM data complicates consistency management. In PIM system, computational bottlenecks primarily emerge as a result of alleviating memory bottleneck via near-data processing. Therefore, integrating CCA directly within PIM is generally more efficient than relying on external accelerators, unless the computational bottleneck arises at the kernel granularity.

Operational Coverage Similar to CGRA, CCA is not fully reconfigurable, as it supports a predefined set of functionalities once fabricated. However, PIM workloads are typically memory-intensive, and the bottleneck within PIM primarily arises from the limited computational capabilities of the PIM processor. Thus, hot codes tend to converge to a small set of operations. For example, in UPMEM, multiplication is decomposed into multiple mul_step instructions, making it a major bottleneck. Accordingly, these operations are consistently identified as dominant bottleneck across most workloads. CCA effectively alleviates these bottlenecks by complementing the limited compute capability through constrained flexibility that specifically targets key operations. Furthermore, we present the PIM-CCA compiler and logic design builder (Section 3.3.3 and 3.3.4), which efficiently leverage hot codes extracted from target PIM workloads. Therefore, this approach is broadly applicable beyond simplistic environments, where non-multiplication patterns are only effective for CCA acceleration.

Directly implementing floating-point (FP) units with CCA can reduce the overhead of software-based emulation in designs that do not natively support FP operations, such as UPMEM. However, this approach introduces several challenges. First, FP units require greater area and power consumption compared to integer (INT) units. Additionally, more complex control logic is necessary to support both FP and INT operations, along with supplementary circuitry to ensure IEEE-754 compliance. Furthermore, increased thermal overhead from FP unit may compromise data integrity within the DRAM cells. For example, HBM2-PIM [19] and GDDR6-AiM [23] integrate native FP units but are limited to fixed-function support for low-precision formats (e.g., FP16/BF16), and simple arithmetic operations (e.g., MAC/ACC). Thus, integrating FP units into PIM remains challenging due to its inherent memory-centric constraints. Even when it is feasible, such integration may degrade PIM performance and raise concerns regarding accuracy and thermal reliability. Similarly, enabling native multiplication via CCA in UPMEM faces challenges comparable to those of native FP support. Nonetheless, as future memory technologies evolve and overcome current limitations, it is expected that more complex units could be employed with CCA, further enhancing the computational capability of PIM architectures.

6 Related Work

Several studies have proposed to improve the performance of PIM architectures. PIMnet [37] addresses the absence of direct DPU-to-DPU links in the UPMEM architecture by introducing a multi-tier, PIM-managed interconnection fabric spanning banks and chips. This removes the host CPU from the data path and enables true in-memory collective communication. DIMM-Link [46] tackles the lack of module-level connectivity in DIMM-based PIM systems by inserting lightweight routing logic and a high-speed, hop-by-hop link between neighboring DIMMs. It reuses standard memory commands to provide PIM cores with a transparent point-to-point and broadcast channel that bypasses both the CPU and dedicated interconnects. ABC-DIMM [38] targets the broadcast pattern itself by embedding an in-channel broadcast protocol and overlapping data distribution with computation. This allows one memory bank to disseminate data to all others without stalling ongoing PIM execution. PID-Comm [33] removes the CPU bottleneck in inter-PE communication on commodity PIM DIMMs by mapping PEs onto a hypercube topology and providing a compact library of collective primitives that execute entirely in memory, enabling efficient point-to-point and group communication across diverse workloads. While prior studies [4, 28, 34] have primarily focused on alleviating memory bottlenecks in PIM systems, the impact of offloaded workload characteristics and the resulting computational bottlenecks within PIM remains largely unexplored. To the best of our knowledge, this is the first work to address these issues.

7 Conclusion

In this work, we proposed the PIM-CCA architecture with a Compute Configurable Accelerator (CCA), specifically tailored for PIM-enabled systems by taking into account workload characteristics and the wiring constraints inherent to PIM architectures. By introducing CCA instruction, pipeline and PIM-CCA Compiler, PIM-CCA effectively accelerates multiplication-intensive patterns that are prevalent in modern workloads, while ensuring lightweight hardware integration. Our evaluation shows up to 1.55x, and a geometric mean of 1.26x performance improvement in compute-intensive workloads, while demonstrates that tasklet count tuning can significantly reduce control overhead in memory-intensive workloads with minimal performance loss. These insights and results highlight the importance of workload- and wire-aware CCA design for future scalable and efficient in-memory computing that meet computation performance requirements.

Acknowledgments

We thank Changyu Seong, Beomseok Kim, and Dongsuk Jeon for their help with the evaluation. This work was supported by Institute of Information & Communications Technology Planning & Evaluation (IITP) grant (2022-0-00441, RS-2020-II201361, RS-2025-02304183), and by the National Research Foundation of Korea(NRF) grant (2022R1A4A1032361, RS-2025-00560614) funded by the Korea government(MSIT). The EDA tool was supported by the IC Design Education Center (IDEC), South Korea. Yongjun Park is the corresponding author.

References

[1] Hadi Asghari-Moghaddam, Young Hoon Son, Jung Ho Ahn, and Nam Sung Kim. 2016. Chameleon: Versatile and practical near-DRAM acceleration architecture for large memory systems. In *2016 49th Annual IEEE/ACM International Symposium on Microarchitecture (MICRO)*. 1–13. https://doi.org/10.1109/MICRO.2016.7783753

[2] Antonio Barbalace, Martin Decky, Javier Picorel, and Pramod Bhatotia. 2020. blockNDP: Block-storage Near Data Processing. In *Proceedings of the 21st International Middleware Conference Industrial Track* (Delft, Netherlands) *(Middleware '20)*. Association for Computing Machinery, New York, NY, USA, 8–15. https://doi.org/10.1145/3429357.3430519

[3] Amirali Boroumand, Saugata Ghose, Youngsok Kim, Rachata Ausavarungnirun, Eric Shiu, Rahul Thakur, Daehyun Kim, Aki Kuusela, Allan Knies, Parthasarathy Ranganathan, and Onur Mutlu. 2018. Google Workloads for Consumer Devices: Mitigating Data Movement Bottlenecks. In *Proceedings of the Twenty-Third International Conference on Architectural Support for Programming Languages and Operating Systems* (Williamsburg, VA, USA) *(ASPLOS '18)*. Association for Computing Machinery, New York, NY, USA, 316–331. https://doi.org/10.1145/3173162.3173177

[4] Jinfan Chen, Juan Gómez-Luna, Izzat El Hajj, Yuxin Guo, and Onur Mutlu. 2023. SimplePIM: A Software Framework for Productive and Efficient Processing-in-Memory. In *2023 32nd International Conference on Parallel Architectures and Compilation Techniques (PACT)*. 99–111. https://doi.org/10.1109/PACT58117.2023.00017

[5] Zeshan Chishti and Berkin Akin. 2019. Memory system characterization of deep learning workloads. In *Proceedings of the International Symposium on Memory Systems* (Washington, District of Columbia, USA) *(MEMSYS '19)*. Association for Computing Machinery, New York, NY, USA, 497–505. https://doi.org/10.1145/3357526.3357569

[6] N. Clark, J. Blome, M. Chu, S. Mahlke, S. Biles, and K. Flautner. 2005. An architecture framework for transparent instruction set customization in embedded processors. In *32nd International Symposium on Computer Architecture (ISCA'05)*. 272–283. https://doi.org/10.1109/ISCA.2005.9

[7] N. Clark, Hongtao Zhong, and S. Mahlke. 2003. Processor acceleration through automated instruction set customization. In *Proceedings. 36th Annual IEEE/ACM International Symposium on Microarchitecture, 2003. MICRO-36.* 129–140. https://doi.org/10.1109/MICRO.2003.1253189

[8] R.H. Dennard, F.H. Gaensslen, Hwa-Nien Yu, V.L. Rideout, E. Bassous, and A.R. LeBlanc. 1974. Design of ion-implanted MOSFET's with very small physical dimensions. *IEEE Journal of Solid-State Circuits* 9, 5 (1974), 256–268. https://doi.org/10.1109/JSSC.1974.1050511

[9] Fabrice Devaux. 2019. The true Processing In Memory accelerator . In *2019 IEEE Hot Chips 31 Symposium (HCS)*. IEEE Computer Society, Los Alamitos, CA, USA, 1–24. https://doi.org/10.1109/HOTCHIPS.2019.8875680

[10] Ivan Fernandez, Ricardo Quislant, Eladio Gutiérrez, Oscar Plata, Christina Giannoula, Mohammed Alser, Juan Gómez-Luna, and Onur Mutlu. 2020. NATSA: A Near-Data Processing Accelerator for Time Series Analysis. In *2020 IEEE 38th International Conference on Computer Design (ICCD)*. 120–129. https://doi.org/10.1109/ICCD50377.2020.00035

[11] Saransh Gupta, Mohsen Imani, Harveen Kaur, and Tajana Simunic Rosing. 2019. NNPIM: A Processing In-Memory Architecture for Neural Network Acceleration. *IEEE Trans. Comput.* 68, 9 (2019), 1325–1337. https://doi.org/10.1109/TC.2019.2903055

[12] Saransh Gupta, Mohsen Imani, Behnam Khaleghi, Venkatesh Kumar, and Tajana Rosing. 2019. RAPID: A ReRAM Processing in-Memory Architecture for DNA Sequence Alignment. In *2019 IEEE/ACM International Symposium on Low Power Electronics and Design (ISLPED)*. 1–6. https://doi.org/10.1109/ISLPED.2019.8824830

[13] Juan Gómez-Luna, Izzat El Hajj, Ivan Fernandez, Christina Giannoula, Geraldo F. Oliveira, and Onur Mutlu. 2022. Benchmarking a New Paradigm: Experimental Analysis and Characterization of a Real Processing-in-Memory System. *IEEE Access* 10 (2022), 52565–52608. https://doi.org/10.1109/ACCESS.2022.3174101

[14] Mingxuan He, Choungki Song, Ilkon Kim, Chunseok Jeong, Seho Kim, Il Park, Mithuna Thottethodi, and T. N. Vijaykumar. 2020. Newton: A DRAM-maker's Accelerator-in-Memory (AiM) Architecture for Machine Learning. In *2020 53rd Annual IEEE/ACM International Symposium on Microarchitecture (MICRO)*. 372–385. https://doi.org/10.1109/MICRO50266.2020.00040

[15] Wenqin Huangfu, Xueqi Li, Shuangchen Li, Xing Hu, Peng Gu, and Yuan Xie. 2019. MEDAL: Scalable DIMM based Near Data Processing Accelerator for DNA Seeding Algorithm. In *Proceedings of the 52nd Annual IEEE/ACM International Symposium on Microarchitecture* (Columbus, OH, USA) *(MICRO '52)*. Association for Computing Machinery, New York, NY, USA, 587–599. https://doi.org/10.1145/3352460.3358329

[16] Bongjoon Hyun, Taehun Kim, Dongjae Lee, and Minsoo Rhu. 2024. Pathfinding Future PIM Architectures by Demystifying a Commercial PIM Technology. In *2024 IEEE International Symposium on High-Performance Computer Architecture (HPCA)*. 263–279. https://doi.org/10.1109/HPCA57654.2024.00029

[17] Donghyeon Kim, Taehoon Kim, Inyong Hwang, Taehyeong Park, Hanjun Kim, Youngsok Kim, and Yongjun Park. 2023. Virtual PIM: Resource-Aware Dynamic DPU Allocation and Workload Scheduling Framework for Multi-DPU PIM Architecture. In *2023 32nd International Conference on Parallel Architectures and Compilation Techniques (PACT)*. 112–123. https://doi.org/10.1109/PACT58117.2023.00018

[18] Hyeoncheol Kim, Taehoon Kim, Taehyeong Park, Donghyeon Kim, Yongseung Yu, Hanjun Kim, and Yongjun Park. 2025. Accelerating LLMs using an Efficient GEMM Library and Target-Aware Optimizations on Real-World PIM Devices. In *Proceedings of the 23rd ACM/IEEE International Symposium on Code Generation and Optimization* (Las Vegas, NV, USA) *(CGO '25)*. Association for Computing Machinery, New York, NY, USA, 225–240. https://doi.org/10.1145/3696443.3708953

[19] Jin Hyun Kim, Shin-haeng Kang, Sukhan Lee, Hyeonsu Kim, Woongjae Song, Yuhwan Ro, Seungwon Lee, David Wang, Hyunsung Shin, Bengseng Phuah, Jihyun Choi, Jinin So, YeonGon Cho, JoonHo Song, Jangseok Choi, Jeonghyeon Cho, Kyomin Sohn, Youngsoo Sohn, Kwangil Park, and Nam Sung Kim. 2021. Aquabolt-XL: Samsung HBM2-PIM with in-memory processing for ML accelerators and beyond. In *2021 IEEE Hot Chips 33 Symposium (HCS)*. 1–26. https://doi.org/10.1109/HCS52781.2021.9567191

[20] Patrycja Krawczuk, George Papadimitriou, Ryan Tanaka, Tu Mai Anh Do, Srujana Subramanya, Shubham Nagarkar, Aditi Jain, Kelsie Lam, Anirban Mandal, Loïc Pottier, and Ewa Deelman. 2021. A Performance Characterization of Scientific Machine Learning Workflows. In *2021 IEEE Workshop on Workflows in Support of Large-Scale Science (WORKS)*. 58–65. https://doi.org/10.1109/WORKS54523.2021.00013

[21] Daehan Kwon, Seongju Lee, Kyuyoung Kim, Sanghoon Oh, Joonhong Park, Gi-Moon Hong, Dongyoon Ka, Kyudong Hwang, Jeongje Park, Kyeongpil Kang, Jungyeon Kim, Junyeol Jeon, Nahsung Kim, Yongkee Kwon, Vladimir Kornijcuk, Woojae Shin, Jongsoon Won, Minkyu Lee, Hyunha Joo, Haerang Choi, Guhyun Kim, Byeongju An, Jaewook Lee, Donguc Ko, Younggun Jun, Ilwoong Kim, Choungki Song, Ilkon Kim, Chanwook Park, Seho Kim, Chunseok Jeong, Euicheol Lim, Dongkyun Kim, Jieun Jang, Il Park, Junhyun Chun, and Joohwan Cho. 2023. A 1ynm 1.25V 8Gb 16Gb/s/Pin GDDR6-Based Accelerator-in-Memory Supporting 1TFLOPS MAC Operation and Various Activation Functions for Deep Learning Application. *IEEE Journal of Solid-State Circuits* 58, 1 (2023), 291–302. https://doi.org/10.1109/JSSC.2022.3200718

[22] Youngeun Kwon and Minsoo Rhu. 2018. Beyond the Memory Wall: A Case for Memory-Centric HPC System for Deep Learning. In *2018 51st Annual IEEE/ACM International Symposium on Microarchitecture (MICRO)*. 148–161. https://doi.org/10.1109/MICRO.2018.00021

[23] Yongkee Kwon, Kornijcuk Vladimir, Nahsung Kim, Woojae Shin, Jongsoon Won, Minkyu Lee, Hyunha Joo, Haerang Choi, Guhyun Kim, Byeongju An, Jeongbin Kim, Jaewook Lee, Ilkon Kim, Jaehan Park, Chanwook Park, Yosub Song, Byeongsu Yang, Hyungdeok Lee, Seho Kim, Daehan Kwon, Seongju Lee, Kyuyoung Kim, Sanghoon Oh, Joonhong Park, Gimoon Hong, Dongyoon Ka, Kyudong Hwang, Jeongje Park, Kyeongpil Kang, Jungyeon Kim, Junyeol Jeon, Myeongjun Lee, Minyoung Shin, Minhwan Shin, Jaekyung Cha, Changson Jung, Kijoon Chang, Chunseok Jeong, Euicheol Lim, Il Park, Junhyun Chun, and Sk Hynix. 2022. System Architecture and Software Stack for GDDR6-AiM. In *2022 IEEE Hot Chips 34 Symposium (HCS)*. 1–25. https://doi.org/10.1109/HCS55958.2022.9895629

[24] Chris Lattner and Vikram Adve. 2004. LLVM: A compilation framework for lifelong program analysis & transformation. In *International symposium on code generation and optimization, 2004. CGO 2004.* IEEE, 75–86.

[25] Donghun Lee, Jinin So, MINSEON AHN, Jong-Geon Lee, Jungmin Kim, Jeonghyeon Cho, Rebholz Oliver, Vishnu Charan Thummala, Ravi shankar JV, Sachin Suresh Upadhya, Mohammed Ibrahim Khan, and Jin Hyun Kim. 2022. Improving In-Memory Database Operations with Acceleration DIMM (AxDIMM). In *Proceedings of the 18th International Workshop on Data Management on New Hardware* (Philadelphia, PA, USA) *(DaMoN '22)*. Association for Computing Machinery, New York, NY, USA, Article 2, 9 pages. https://doi.org/10.1145/3533737.3535093

[26] Sukhan Lee, Shin-haeng Kang, Jaehoon Lee, Hyeonsu Kim, Eojin Lee, Seungwoo Seo, Hosang Yoon, Seungwon Lee, Kyounghwan Lim, Hyunsung Shin, Jinhyun Kim, O Seongil, Anand Iyer, David Wang, Kyomin Sohn, and Nam Sung Kim. 2021. Hardware Architecture and Software Stack for PIM Based on Commercial DRAM Technology : Industrial Product. In *2021 ACM/IEEE 48th Annual International Symposium on Computer Architecture (ISCA)*. 43–56. https://doi.org/10.1109/ISCA52012.2021.00013

[27] John Levine. 2009. *Flex & Bison: Text Processing Tools.* " O'Reilly Media, Inc.".

[28] Siyuan Ma, Kaustubh Mhatre, Jian Weng, Bagus Hanindhito, Zhengrong Wang, Tony Nowatzki, Lizy John, and Aman Arora. 2024. PIMSAB: A Processing-In-Memory System with Spatially-Aware Communication and Bit-Serial-Aware Computation. *ACM Trans. Archit. Code Optim.* 21, 4, Article 91 (Nov. 2024), 27 pages. https://doi.org/10.1145/3690824

[29] Vikram Sharma Mailthody, Zaid Qureshi, Weixin Liang, Ziyan Feng, Simon Garcia de Gonzalo, Youjie Li, Hubertus Franke, Jinjun Xiong, Jian Huang, and Wen-mei Hwu. 2019. DeepStore: In-Storage Acceleration for Intelligent Queries. In *Proceedings of the 52nd Annual IEEE/ACM International Symposium on Microarchitecture* (Columbus, OH, USA) *(MICRO '52)*. Association for Computing Machinery, New York, NY, USA, 224–238. https://doi.org/10.1145/3352460.3358320

[30] Hosein Mohammadi Makrani, Hossein Sayadi, Sai Manoj Pudukotai Dinakarra, Setareh Rafatirad, and Houman Homayoun. 2018. A comprehensive memory analysis of data intensive workloads on server class architecture. In *Proceedings of the International Symposium on Memory Systems* (Alexandria, Virginia, USA) *(MEMSYS '18)*. Association for Computing Machinery, New York, NY, USA, 19–30. https://doi.org/10.1145/3240302.3240320

[31] Nika Mansouri Ghiasi, Jisung Park, Harun Mustafa, Jeremie Kim, Ataberk Olgun, Arvid Gollwitzer, Damla Senol Cali, Can Firtina, Haiyu Mao, Nour Almadhoun Alserr, Rachata Ausavarungnirun, Nandita Vijaykumar, Mohammed Alser, and Onur Mutlu. 2022. GenStore: A High-Performance in-Storage Processing System for Genome Sequence Analysis. In *Proceedings of the 27th ACM International Conference on Architectural Support for Programming Languages and Operating Systems* (Lausanne, Switzerland) *(ASPLOS '22)*. Association for Computing Machinery, New York, NY, USA, 635–654. https://doi.org/10.1145/3503222.3507702

[32] Onur Mutlu and Lavanya Subramanian. 2014. Research problems and opportunities in memory systems. *Supercomputing frontiers and innovations* 1, 3 (2014), 19–55.

[33] Si Ung Noh, Junguk Hong, Chaemin Lim, Seongyeon Park, Jeehyun Kim, Hanjun Kim, Youngsok Kim, and Jinho Lee. 2024. PID-Comm: A Fast and Flexible Collective Communication Framework for Commodity Processing-in-DIMM Devices. In *2024 ACM/IEEE 51st Annual International Symposium on Computer Architecture (ISCA)*. 245–260. https://doi.org/10.1109/ISCA59077.2024.00027

[34] Seyyed Hossein SeyyedAghaei Rezaei, Mehdi Modarressi, Rachata Ausavarungnirun, Mohammad Sadrosadati, Onur Mutlu, and Masoud Daneshtalab. 2020. NoM: Network-on-Memory for Inter-Bank Data Transfer in Highly-Banked Memories. *IEEE Computer Architecture Letters* 19, 1 (2020), 80–83. https://doi.org/10.1109/LCA.2020.2990599

[35] Zhenyuan Ruan, Tong He, and Jason Cong. 2019. INSIDER: Designing In-Storage Computing System for Emerging High-Performance Drive. In *2019 USENIX Annual Technical Conference (USENIX ATC 19)*. USENIX Association, Renton, WA, 379–394. https://www.usenix.org/conference/atc19/presentation/ruan

[36] Paulo Cesar Santos, Francis Birck Moreira, Aline Santana Cordeiro, Sairo Raoní Santos, Tiago Rodrigo Kepe, Luigi Carro, and Marco Antonio Zanata Alves. 2021. Survey on near-data processing: Applications and architectures. *Journal of Integrated Circuits and Systems* 16, 2 (2021), 1–17. https://doi.org/10.29292/jics.v16i2.502

[37] Hyojun Son, Gilbert Jonatan, Xiangyu Wu, Haeyoon Cho, Kaustubh Shivdikar, José L. Abellán, Ajay Joshi, David Kaeli, and John Kim. 2025. PIMnet: A Domain-Specific Network for Efficient Collective Communication in Scalable PIM. In *2025 IEEE International Symposium on High Performance Computer Architecture (HPCA)*. 1557–1572. https://doi.org/10.1109/HPCA61900.2025.00116

[38] Weiyi Sun, Zhaoshi Li, Shouyi Yin, Shaojun Wei, and Leibo Liu. 2021. ABC-DIMM: Alleviating the Bottleneck of Communication in DIMM-based Near-Memory Processing with Inter-DIMM Broadcast. In *2021 ACM/IEEE 48th Annual International Symposium on Computer Architecture (ISCA)*. 237–250. https://doi.org/10.1109/ISCA52012.2021.00027

[39] Synopsys. [n. d.]. Design Compiler: RTL Synthesis. https://www.synopsys.com/implementation-and-signoff/rtl-synthesis-test/dc-ultra.html.

[40] Dufy Teguia, Jiaxuan Chen, Stella Bitchebe, Oana Balmau, and Alain Tchana. 2024. vPIM: Processing-in-Memory Virtualization. In *Proceedings of the 25th International Middleware Conference* (Hong Kong, Hong Kong) *(Middleware '24)*. Association for Computing Machinery, New York, NY, USA, 417–430. https://doi.org/10.1145/3652892.3700782

[41] Mahdi Torabzadehkashi, Siavash Rezaei, Vladimir Alves, and Nader Bagherzadeh. 2018. CompStor: An In-storage Computation Platform for Scalable Distributed Processing. In *2018 IEEE International Parallel and Distributed Processing Symposium Workshops (IPDPSW)*. 1260–1267. https://doi.org/10.1109/IPDPSW.2018.00195

[42] Mahdi Torabzadehkashi, Siavash Rezaei, Ali Heydarigorji, Hosein Bobarshad, Vladimir Alves, and Nader Bagherzadeh. 2019. Catalina: In-Storage Processing Acceleration for Scalable Big Data Analytics. In *2019 27th Euromicro International Conference on Parallel, Distributed and Network-Based Processing (PDP)*. 430–437. https://doi.org/10.1109/EMPDP.2019.8671589

[43] UPMEM. [n. d.]. Accelerating compute by cramming it into DRAM memory. https://www.upmem.com/nextplatform-com-2019-10-03-accelerating-compute-by-cramming-it-into-dram/#:~:text=The%20first%20generation%20of%20Upmem,and%20Power9%20processors%20are%20currently.

[44] UPMEM. 2022. UPMEM Processing In-Memory (PIM) (Tech Paper).

[45] Sheng Xu, Xiaoming Chen, Xuehai Qian, and Yinhe Han. 2020. TUPIM: A Transparent and Universal Processing-in-Memory Architecture for Unmodified Binaries. In *Proceedings of the 2020 on Great Lakes Symposium on VLSI* (Virtual Event, China) *(GLSVLSI '20)*. Association for Computing Machinery, New York, NY, USA, 199–204. https://doi.org/10.1145/3386263.3406896

[46] Zhe Zhou, Cong Li, Fan Yang, and Guangyu Sun. 2023. DIMM-Link: Enabling Efficient Inter-DIMM Communication for Near-Memory Processing. In *2023 IEEE International Symposium on High-Performance Computer Architecture (HPCA)*. 302–316. https://doi.org/10.1109/HPCA56546.2023.10071005